

Lab 3 Report:

MNIST Classification with FCN

Name:

```
In [233... # Import necessary packages

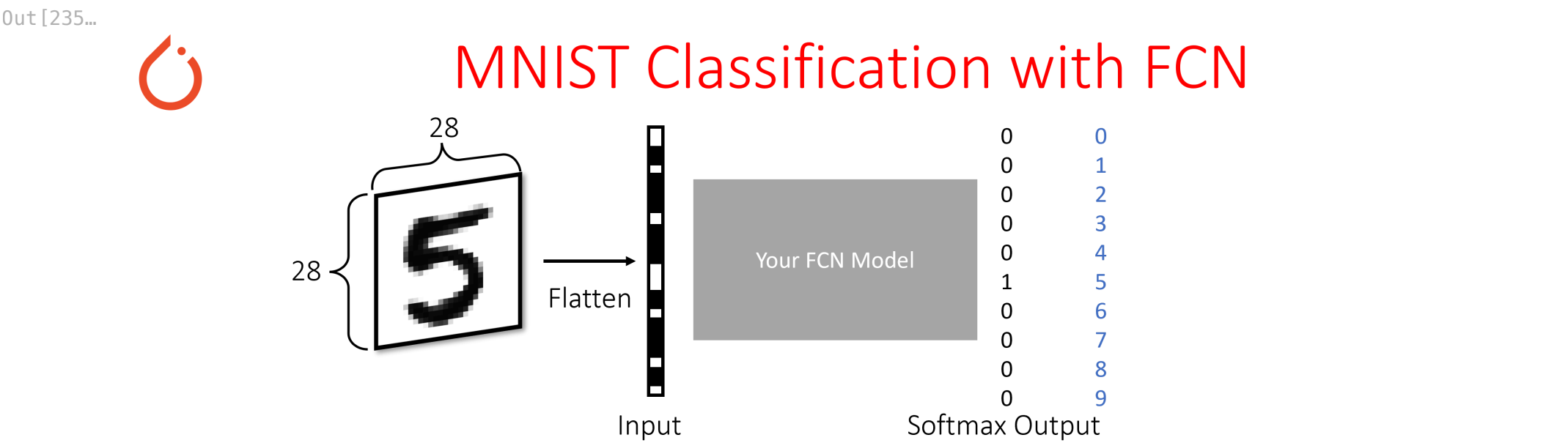
%matplotlib inline

import matplotlib.pyplot as plt

import torch
import torchvision
import numpy as np
```

```
In [234... from IPython.display import Image # For displaying images in colab jupyter cell
```

```
In [235... Image('lab3_exercise.PNG', width = 1000)
```



In this exercise, you will classify handwritten digits (28 x 28) using your own Fully Connected Network Architecture.

Prior to training your neural net, 1) Flatten each digit into 1D array of size 784, 2) Normalize the dataset using standard scaler and 3) Split the dataset into train/validation/test.

Design your own neural net architecture with your choice of hidden layers, activation functions, optimization method etc.

Your goal is to achieve a testing accuracy of >90%, with no restrictions on epochs.

Demonstrate the performance of your model via plotting the training loss, validation accuracy and printing out the testing accuracy.

Plot the testing samples where your model failed to classify correctly and print your model's best guess for each of them

Prepare Data

```
In [236... # Load MNIST Dataset in Numpy

# 1000 training samples where each sample feature is a greyscale image with shape (28, 28)
# 1000 training targets where each target is an integer indicating the true digit
mnist_train_features = np.load('mnist_train_features.npy')
mnist_train_targets = np.load('mnist_train_targets.npy')

# 100 testing samples + targets
mnist_test_features = np.load('mnist_test_features.npy')
mnist_test_targets = np.load('mnist_test_targets.npy')

# Print the dimensions of training sample features/targets
print(mnist_train_features.shape, mnist_train_targets.shape)
# Print the dimensions of testing sample features/targets
print(mnist_test_features.shape, mnist_test_targets.shape)
```

(1000, 28, 28) (1000,)
(100, 28, 28) (100,)

```
In [237... # Let's visualize some training samples

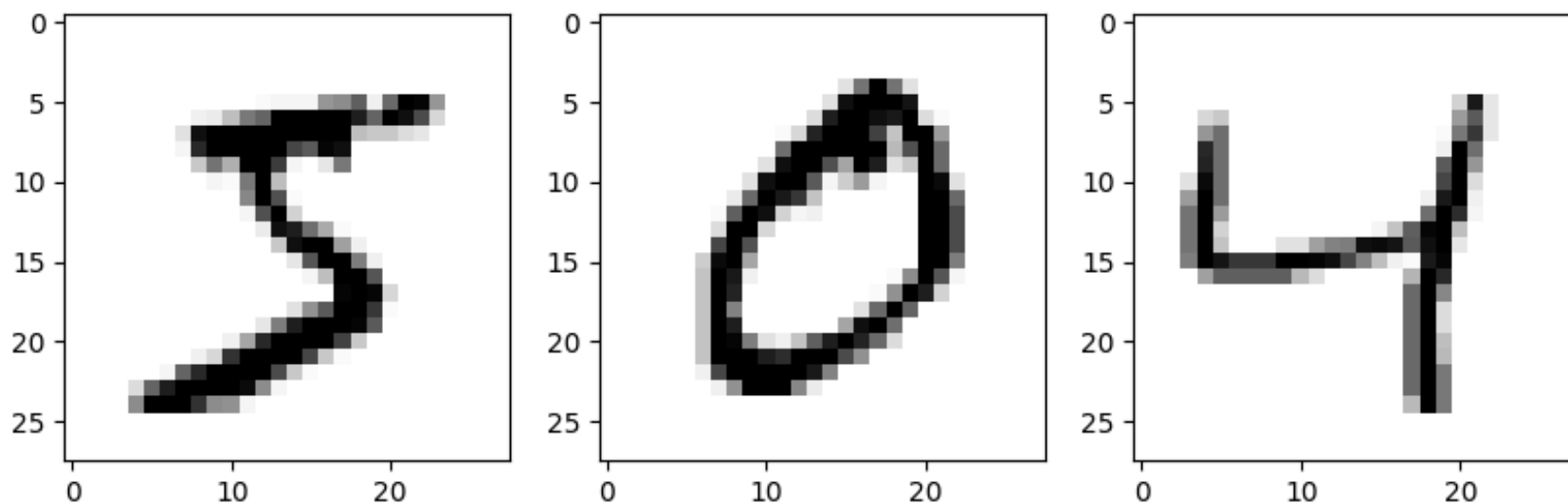
plt.figure(figsize = (10, 10))

plt.subplot(1,3,1)
plt.imshow(mnist_train_features[0], cmap = 'Greys')
```

```
plt.subplot(1,3,2)
plt.imshow(mnist_train_features[1], cmap = 'Greys')

plt.subplot(1,3,3)
plt.imshow(mnist_train_features[2], cmap = 'Greys')
```

Out[237... <matplotlib.image.AxesImage at 0x16d002260>



```
In [238... # Reshape features via flattening the images
# This refers to reshape each sample from a 2d array to a 1d array.
# hint: np.reshape() function could be useful here

mnist_train_features = np.reshape(mnist_train_features, (1000, -1)) # Reshape train features to 1d
mnist_test_features = np.reshape(mnist_test_features, (100, -1)) # Reshape test features to 1d

print(mnist_train_features.shape, mnist_test_features.shape)

(1000, 784) (100, 784)
```

```
In [239... # Scale the dataset according to standard scaling
# Scaling using (x-mu)/std
mnist_train_features = (mnist_train_features - np.mean(mnist_train_features))/np.std(mnist_train_features)
mnist_test_features = (mnist_test_features - np.mean(mnist_test_features))/np.std(mnist_test_features)
```

```
In [240... # Split training dataset into Train (90%), Validation (10%)
split_proportion = [int(0.9 * mnist_train_features.shape[0]), int(1 * mnist_train_features.shape[0])]

# Split validation features and targets according to split proportion
mnist_validation_features = np.split(mnist_train_features, split_proportion, axis = 0)[1]
mnist_validation_targets = np.split(mnist_train_targets, split_proportion, axis = 0)[1]

# Split train features and targets according to split proportion
mnist_train_features = np.split(mnist_train_features, split_proportion, axis = 0)[0]
mnist_train_targets = np.split(mnist_train_targets, split_proportion, axis = 0)[0]
```

Define Model

```
In [ ]: # Define class model
class mnistClassification(torch.nn.Module):

    def __init__(self, input_dim, output_dim):
        """
        This function initialized layers in our FCN
        """
        super(mnistClassification, self).__init__()

        # Set up input layer with input dimension
        self.layer1 = torch.nn.Linear(input_dim, 400)
        self.layer2 = torch.nn.Linear(400, 400) # Hidden layer with 400 neurons
        self.dropout = torch.nn.Dropout(p = 0.3) # Dropout layer with p=0.3
        self.layer3 = torch.nn.Linear(400, 400) # Another hidden layer with 400 neurons
        self.layer4 = torch.nn.Linear(400, output_dim) # Output layer

    def forward(self, x):
        """
        This function passes input through each layers forward and compute output
        """
        out1 = self.layer1(x) # Output from layer 1
        out1 = torch.nn.functional.relu(out1) # Activation function 1 for output 1
        out2 = self.layer2(out1) # Output from layer 2
        out2 = torch.nn.functional.relu(out2) # Activation function 2 for output 2
        out2 = self.dropout(out2) # Dropout layer for output 2
        out3 = self.layer3(out2) # Output from layer 3
        out3 = torch.nn.functional.relu(out3) # Activation function 2 for output 3
        output = self.layer4(out3) # final output layer

        return output
```

Define Hyperparameters

```
In [242... # Initialize our neural network model with input and output dimensions
model = mnistClassification(input_dim = 784, output_dim = 10)

# Define the learning rate and epoch
learning_rate = 0.001
epochs = 400

# Define loss function and optimizer
loss_func = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = learning_rate, weight_decay=0.002)
# # Run this line if you have PyTorch GPU version
# if torch.cuda.is_available():
#     model.cuda()

model
```

```
Out[242... mnistClassification(
  (layer1): Linear(in_features=784, out_features=400, bias=True)
  (layer2): Linear(in_features=400, out_features=400, bias=True)
  (dropout): Dropout(p=0.3, inplace=False)
  (layer3): Linear(in_features=400, out_features=400, bias=True)
  (layer4): Linear(in_features=400, out_features=10, bias=True)
)
```

Identify Tracked Values

```
In [243... mnist_train_features.shape
```

```
Out[243... (900, 784)
```

```
In [244... # Placeholders for training loss and validation accuracy during training
# Training loss should be tracked for each iteration (1 iteration -> single forward pass to the network)
# Validation accuracy should be evaluated every 'Epoch' (1 epoch -> full training dataset)
# If using batch gradient, 1 iteration = 1 epoch

train_loss_list = np.zeros((epochs,)) # Because using batch gradient 1 it = 1 epoch
validation_accuracy_list = np.zeros((epochs,))
```

Train Model

```
In [245... import tqdm

# Convert the training, validation, testing dataset (NumPy arrays) into torch tensors

train_inputs = torch.from_numpy(mnist_train_features).float()
train_targets = torch.from_numpy(mnist_train_targets).long()

validation_inputs = torch.from_numpy(mnist_validation_features).float()
validation_targets = torch.from_numpy(mnist_validation_targets).long()

test_inputs = torch.from_numpy(mnist_test_features).float()
test_targets = torch.from_numpy(mnist_test_targets).long()

# Training Loop -----
for epoch in tqdm.trange(epochs):

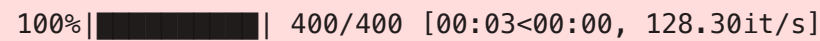
    # start training
    model.train()
    train_outputs = model(train_inputs)
    loss = loss_func(train_outputs, train_targets) # Feed forward train outputs to compute loss against targets
    train_loss_list[epoch] = loss.item()

    # Zero gradients
    optimizer.zero_grad()
    loss.backward() # Adjust network weights and biases
    optimizer.step() #according to optimizer

    # Compute Validation Accuracy -----

    with torch.no_grad():
        model.eval() # Evaluating model
        validation_outputs = model(validation_inputs) # Pass validation inputs to keep track of validation accuracy
        correct = (torch.argmax(validation_outputs, dim=1) ==
        validation_targets).type(torch.FloatTensor) # Compute correct situations

        validation_accuracy_list[epoch] = correct.mean() # Compute accuracy
```



Visualize and Evaluate Model

```
In [246... # Import seaborn for prettier plots
```

```
import seaborn as sns
```

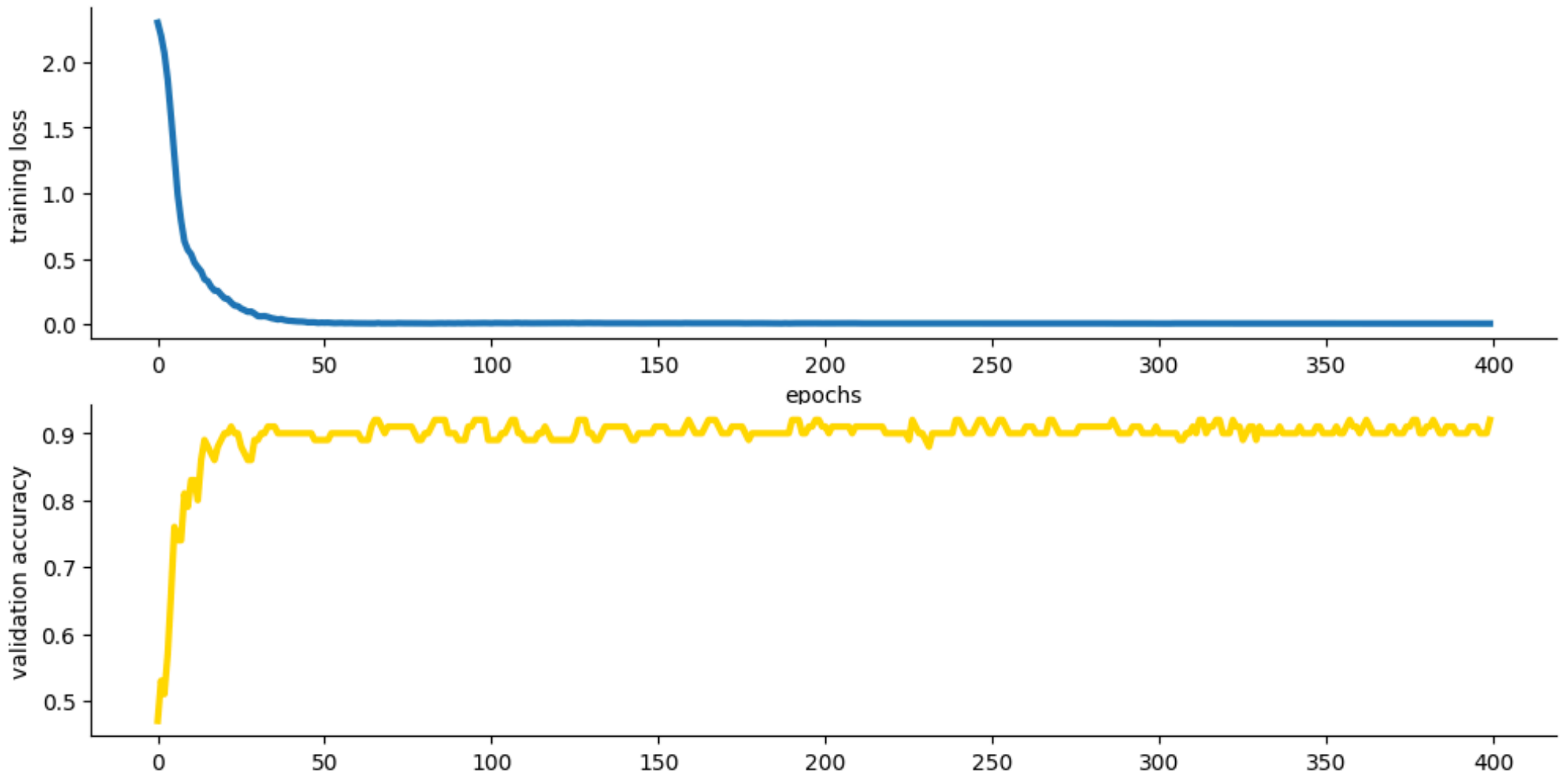
In [247...

```
# Visualize training loss

plt.figure(figsize = (12, 6))

# Visualize training loss with respect to iterations (1 iteration -> single batch)
plt.subplot(2, 1, 1)
plt.plot(train_loss_list, linewidth = 3)
plt.ylabel("training loss")
plt.xlabel("epochs")
sns.despine()

# Visualize validation accuracy with respect to epochs
plt.subplot(2, 1, 2)
plt.plot(validation_accuracy_list, linewidth = 3, color = 'gold')
plt.ylabel("validation accuracy")
sns.despine()
```



In [248...

```
# Compute the testing accuracy

with torch.no_grad():
    model.eval() # Start evaluation
    y_pred_test = model(test_inputs) # Compute predicted test output using test input
    correct = (torch.argmax(y_pred_test, dim=1) == test_targets).type(torch.FloatTensor) # Find correctness
    print("Testing Accuracy:" + str(correct.mean().numpy())) # Compute accuracy
```

Testing Accuracy:0.91

In []:

```
# Plot 5 incorrectly classified testing samples and print the model predictions for each of them
# You can use np.reshape() to convert flattened 1D array back to 2D array

with torch.no_grad():
    y_pred_test = model(test_inputs) # Compute predicted test output using test input
    incorrect = (torch.argmax(y_pred_test, dim=1) != test_targets) # Find incorrect situations
    idx = torch.where(incorrect)[0] # Find index
    idx = idx[:5] # Select first 5 indices

# Reshape testing samples to 2d
mnist_test_features = mnist_test_features.reshape(-1, 28, 28)
preds = torch.argmax(y_pred_test, dim=1) # Find predictions
plt.figure(figsize=(12, 6)) # Start a figure

# Plot 5 incorrectly classified testing samples
for pos, i in enumerate(idx):
    plt.subplot(1, len(idx), pos+1)
    plt.imshow(mnist_test_features[i], cmap = 'Greys')
    plt.title(f"Predicted:{preds[i].item()}") # Print out model predictions for each of them
```

